Betriebssysteme

Ergänzungen zur Übung 3

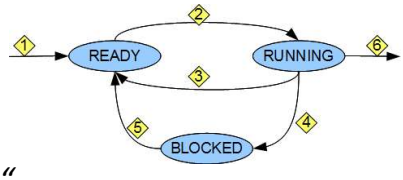
Prof. Dr. Harald, Kornmayer
Institut für Informatik, DHBW Mannheim, Germany

Übung 2

- Systemaufrufe
  - fork()
  - exec()
  - (siehe auch Aufgabenblatt 2)
- POSIX Thread library
- JAVA Threads

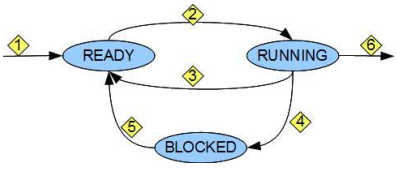
Systemaufrufe

- fork()
  - Erzeugung von Processen
  - man 2 fork
  - *„creates a new process by duplicating the calling process“*
  - *benötigt **#include** <unistd.h> (**#include** <sys/types.h> für pid_t)*
  - *Der Rückgabewert unterscheidet zwischen Kind und Vater*
    - *Fehlerfall: -1*
    - *Vater: > 0*
    - *Kind: 0*



Systemaufrufe

- exec()
  - Erzeugung von Prozessen
    - führt ein andere Datei aus!
    - mit neuen Code!
- 6 Varianten
  - Argumente als Liste: execl, execle, execlp
  - Argumente als Vector: execv, execve, execvp
    - „e“ - Environment
    - „p“ - Path
  - Bsp: execlp(prog_name, prog_name, Arg1, Arg2, … 0) ;
    - execlp(„sort“, „sort“, „-n“, „studentenliste“, 0) ;
    - „sort“ wird aus historischen Gründen zweimal angegeben!
    - 0 terminiert die Eingabe Liste



Systemaufrufe

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
int main(void) {
  printf("Schau mer mal"); /* prints Schau mer mal! */
  int pid1, pid2;
  pid1 = getpid();
  printf(" Process ID %d", pid1);

  pid2 = fork();
  if (pid2 < 0) {
     perror("\n Fork war nicht erfolgreich");
    exit(1);
  }
  else if (pid2 == 0) {
     // Kind prozess
     printf("\nKIND Prozess");
  }
  else if (pid2 > 0) {
    // Vater prozess
    printf("\nVATER Prozess ");
  }
  return EXIT_SUCCESS;
}
```


JAVA Threads

- Integraler Bestandteil der Sprache
- Interface Runable bzw. Klasse Thread
  - Überschreiben der Methode run()
- Weitere Methoden
  - start(): ein Thread wird gestartet
    - Beendet sich mit dem Ende der Methode run()
  - yield(): ein Thread gibt den Prozessor freiwillig ab. Der Scheduler verteilt anschliessend die CPU an wartende Threads
  - sleep(): Unterbrechung des Threads für eine vorgegebene Zeit
  - join(): warten auf Terminierung eines Threads
  - wait(): Thread wartet auf eine Benachrichtigung
  - notify(), notifyAll(): Versenden einer Benachrichtigung
  - synchronized: Lock-Mechanismus (später: Semaphore)

POSIX Threads

- POSIX = Portable Operating System Interface
  - Standardisiertes (API) Application Programming Interface für UNIX-artige Systems
  - Umfasst
    - Systemaufrufe und
    - Kommandozeileninterpreter und Hilfsprogramme
    - Schnittstelle zwischen Anwendung und Betriebssystem
  - Threads sind ein Beispiel für ein API
    - IEEE POSIX 1003.1c
    - Implementierungen werden als POSIX threads oder Pthreads bezeichnet

POSIX Threads

- C-Library
  - #include <pthread.h> und „gcc .... -lthread ...“
- > 60 Funktionsaufrufe
- Wichtigste Befehle

– pthread_create	Erzeugt neuen Thread
– pthread_exit	Beendet den aufrufenden Thread
– pthread_join	Wartet auf die Beendigung eines bestimmten Threads
– pthread_yield	Gibt CPU frei, damit andere Threads laufen können
– pthread_attr_init	Erzeugt und initialisiert eine Attributsstruktur
– pthread_attr_destroy	Löscht die Attributsstruktur

- Unter Linux u.U. Probleme mit Manpages → Web!

Aufgabe

- Übungsblatt 3
- Siehe Moodle-Webseite!

  – Viel Spass beim Programmieren!